

A Digit Extraction Algorithm for the Eulers Constant γ : On the Irrationality of the Euler Constant

γ

M. Chowdhury
5th March 2025

Abstract

Up to now there is no digit extraction algorithm for the Euler Constant γ . A first digit extraction algorithm is given for the Euler Constant $\gamma = 0.577216\dots$ that computes the p th binary digit and it does not require to compute the previous digits prior to the p th to compute the p th digit. Our digit extraction algorithm for γ requires $O(p^2)$ space at most and $O(p^2)$ time at most. We think it is surprising that the time is not exponential in p which is one reason why we think our algorithm is interesting. The algorithm is based on a variant of the known BBP digit extraction algorithm that is used to extract digits various constants such as π or $\ln 2$, various values of the Riemann Zeta function and more constants. One limitation of our algorithm is that it can only extraction digits at certain positions in the expansion of γ but the number of positions is infinite. Our algorithm gives a new way to prove γ is irrational by considering the behaviour of the equations or algorithm is based on. The algorithm uses only arbitrary precision integer arithmetic and requires no need to compute transcendental function which makes it suitable to compute on embedded CPUs or microcontrollers with low resources.

1 Introduction

Embedded CPUs or MCUs, such as the (8 bit) Intel 8008 or an (4 bit) Intel 4004, have very restricted arithmetic operation which may only be addition or subtraction of 8 bit length numbers and they do not have hardware for floating point numbers. We present a digit extraction algorithm for the Euler Constant $\gamma = 0.577216\dots$, suitable for use with the limited mathematics operations of embedded CPUs or MCUs, which we

think works surprisingly fast: in linear time, $O(d)$ of the digits position d and in $O(d)$ linear space for the digit position d . This is comparable, to the algorithmic complexity of a different constant π and different algorithm, which is the spigot algorithm for π by Rabinowitz-Wagon [1] which also require linear time, $O(d)$ of the digits position d and in $O(d)$ linear space for the digit position d for computing digits of π -but this π algorithm requires the use of computing the prior digits to the d th digit to compute the d th digit. The above π algorithm requires arbitrary precision arithmetic because to compute the d th position, in the above π algorithm, takes $O(d)$ bits. Early interest in spigot algorithm came from considering such computations on CPUs such as the Intel 8008 or Intel 4004. Our algorithm can be implemented using integer arithmetic only.

Unlike the above π spigot algorithm our digit extraction algorithm for γ does not require to use or compute the digits prior to the d th digit of γ to compute the d th digit of γ . But like the above π spigot algorithm it also requires linear time, $O(d)$ in the digits position d and in $O(d)$ linear space for extracting digits and requires arbitrary precision arithmetic. Actually the fastest algorithm to compute π (by Chudnovsky et al) which was used to break the world record for the most computed digits of π also requires linear time, $O(d)$ of the digits position d and in $O(d)$ linear space for the digit position d for computing digits and arbitrary precision arithmetic-but the Chudnovsky is algorithm is not a digit extraction algorithm.

The overall idea in our new algorithm create an algorithm based on the BBP (Borwien-Bailey-Plouffe) digit extraction algorithm for γ using our new ideas. In general for a constant c a BBP series is a series of the form

$$c = \sum_{u=0}^{\infty} \frac{p(u)}{q(u)b^u}$$

where $p(u)$ and $q(u)$ are integer valued polynomials, in u , and $\deg(p) < \deg(q)$ and $q(u)$ is non-zero for $u \geq 0$. A BBP algorithm uses such an above BBP series above to compute individual digits of some constant c (e.g. c may equal to π or $\ln 2$ or some integer values N of $\zeta(N)$) and it involves using a binary exponentiation algorithm (to speed up the computation of b^u above) by using the binary expansion of u and it involves using the fact that the series error is geometric so the number of terms to compute is known before hand. The BBP algorithm only requires a few memory elements.

2 A Digit Extraction Algorithm for γ

In this section we describe our new algorithm. In [2] the following series is given for Eulers constant which converges with error $O(2^{-k})$

$$\gamma = 1 - \log k \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} + \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2} + O(2^{-k}) \quad (1)$$

This converges with geometric error to compute the k th binary digit of γ which is the correct type of error growth required for our algorithm in this section to work.

Thus, assuming we have infinite precision fast arithmetic-which we do not, to compute the d th digit of γ we compute, where $d = k - 1$

$$\gamma = (2^d (1 - (\log k) \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} + \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2})) \bmod 1 \quad (2)$$

While equation 2 does not have the form of BBP series since the $\log k$ for any value of k is the problem here multiplied by some term from the finite sum. But we can convert it into a BBP like formula, that uses arbitrary precision arithmetic, the first step is to note if $\log k$ has a BBP series and substitute that series into 2 as the starting point to derive our new algorithm. The value of k in $\log k$ term of equation 2 determines what base we can extract the digits of γ in. Note that there is a BBP series for $\log k$ when k is a certain prime (but it is not known if this is true for an infinite number of primes but we think probably likely to be true)-this is based of Aurefeuille's factorisations [4]. Considering a version of our algorithm where k is some prime is something we will do in a future paper. So for now we take a different route to derive our algorithm. So note that there is a BBP series for $\log k$ when k power of 2, one way this can be proved by using the BBP series for $\log 2$ (natural logarithm of 2) which is

$$\ln 2 = \sum_{j=1}^{\infty} \frac{1}{j 2^j}$$

then

$$\ln 2^g = \sum_{j=1}^{\infty} \frac{2^g}{j 2^j} \quad (3)$$

for any integer g . Now assume here we want to compute the k th where

$k = 2^g$. Substituting 3 into 2 gives

$$\gamma = 2^d(1 - (\log k) \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} + \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2}) \bmod 1 \quad (5)$$

$$= (2^d(1 - [(\log 2^g) \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} + \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2}]) \bmod 1) \quad (6)$$

$$= (2^d(1 - [(\sum_{j=1}^{\infty} \frac{2^g}{j2^j}) \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} + \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2}]) \bmod 1) \quad (7)$$

$$= (2^d(1 - [(\sum_{j=1}^{\infty} \frac{2^g}{j2^j}) \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} + \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2}]) \bmod 1) \quad (8)$$

$$= (1 - [(\sum_{j=1}^{\infty} (\frac{2^{g+d}}{j2^j}) \sum_{r=1}^{r=12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} \bmod 1 \quad (9)$$

$$+ \sum_{r=1}^{r=12k+1} \frac{2^d(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2} \bmod 1]) \bmod 1) \\ = (1 - [((\sum_{r=1}^{r=12k+1} \frac{\sum_{j=1}^{\infty} (\frac{2^{g+d}}{j2^j})(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} \bmod 1) \bmod 1) \quad (9.1)$$

$$+ \sum_{r=1}^{r=12k+1} \frac{2^d(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2} \bmod 1]) \bmod 1$$

$$= (1 - (((\sum_{r=1}^{r=12k+1} \frac{\sum_{j=1}^{\infty} (\frac{2^{g+d}}{j2^j})(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} \bmod 1) \bmod 1) \quad (9.1)$$

$$+ \sum_{r=1}^{r=12k+1} \frac{2^d(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)^2} \bmod 1)) \bmod 1$$

Note that the mod 1 operation is done similar to how it is done in the standard BBP algorithm, a term is computed mod 1 then it is added to the next term in the sum computed mod 1 and then this partial sum is computed mod 1 and so on. Of course there is more than one way to compute the summation mod 1. So from equation 9.1 or equation 8 we can now give our algorithm. Note that equation 9.1 contains easily computable integers in the upper limits of the summation, and it uses only integer arithmetic, which makes it suitable to compute on an embedded MCU or embedded CPU. We show that 9.1 (a rewrite of equation 8) can be computed with a BBP like algorithm as follows. Our algorithm only

works where the digit k has an BBP formula for $\ln k$, here k is powers of 2. First we note that

$$(C_2(2^d \log 2^g)) \bmod 1 = (C_2 \sum_{j=1}^{\infty} \frac{2^{g+d}}{j2^j}) \bmod 1 = R + \varepsilon$$

can be computed, or some rational C_2 , with the a BBP like algorithm for some rational R up to some error ε we can get the value of the above to $\varepsilon \leq \frac{1}{2^h}$ for some integer h in for computing the d th digit $\log 2^g$ in $O(d \log(d))$ time. In other words the error can be controlled here to the desired amount, In the standard BBP algorithm it is known that the error can be made to be geometrically small by computing a known number of terms-in other words there are an infinite number of terms in the known BBP algorithm that total less than a "geometric amount" of absolute error- and we use this idea here. The error in the BBP formula occurs after ignoring all the term from s in it

$$c = \sum_{u=0}^{\infty} \frac{p(u)}{q(u)b^u} = \sum_{u=0}^{s-1} \frac{p(u)}{q(u)b^u} + \sum_{u=s}^{\infty} \frac{p(u)}{q(u)b^u}$$

thus the geometric amount of error ϵ is

$$\epsilon = (\sum_{u=s}^{\infty} \frac{p(u)}{q(u)b^u}) \bmod 1$$

and R is

$$R = (\sum_{u=0}^{s-1} \frac{p(u)}{q(u)b^k}) \bmod 1$$

From here we can derive a few way to generate a BBP like algorithm for γ but we give one example below. Firstly consider the left hand term of 9.1 which is

$$(1 - ((\sum_{r=1}^{r=12k+1} \frac{\sum_{j=1}^{\infty} (\frac{2^{g+d}}{j2^j}) (-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} \bmod 1) \bmod 1))$$

we can ignore the subtraction of the 1 in the above as that does not have an impact on the absolute error analysis here.

Then 9.1 is the expansion.

$$\begin{aligned}
& \sum_{j=1}^{\infty} \left(\frac{2^{g+d}}{j2^j} \right) \sum_{r=1}^{12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} \mod 1 \\
&= \left(\left(\sum_{j=1}^{\infty} \left(\frac{2^{g+d}}{j2^j} \right) \frac{(-1)^{1+1} k^{1+1}}{(1-1)!(1+1)} \right) \mod 1 + \sum_{j=1}^{\infty} \left(\frac{2^{g+d}}{j2^j} \right) \frac{(-1)^{2+1} k^{2+1}}{(2-1)!(2+1)} \right) \mod 1 + \\
& \sum_{j=1}^{\infty} \left(\frac{2^{g+d}}{j2^j} \right) \frac{(-1)^{3+1} k^{3+1}}{(3-1)!(3+1)} \mod 1 \dots + \\
& \sum_{j=1}^{\infty} \left(\frac{2^{g+d}}{j2^j} \right) \frac{(-1)^{(12k+1)+1} k^{(12k+1)+1}}{((12k+1)-1)!((12k+1)+1)} \mod 1
\end{aligned} \tag{10}$$

a general term in the above LHS term of 9 is then

$$\begin{aligned}
& \left(\sum_{j=1}^{\infty} \left(\frac{2^{g+d}}{j2^j} \right) \frac{(-1)^{t+1} k^{t+1}}{(t-1)!(t+1)} \right) \mod 1 \\
&= \left(\sum_{j=1}^{\infty} \left(\frac{2^{g+d}}{j2^j} \right) \frac{(-1)^{t+1} k^{t+1}}{(t-1)!(t+1)} \right) \\
& \text{for } 1 \leq t \leq 12k+1 \text{ where } k = 2^d
\end{aligned}$$

we can think of the above as a BBP formula multiplied by a constant term where the constant term is a term in the finite series of the LHS of 9. So we can use a BBP formula here with arbitrary precision arithmetic (since the factorial term in above means we must use arbitrary precision arithmetic here)-we refer to this as the variant BBP algorithm which will be explained more in detail in a further paper. Since

$$\left(\left(\frac{(-1)^{t+1} k^{t+1}}{(t-1)!(t+1)} \right) 2^d \log 2^g \right) \mod 1 = \left(\sum_{j=1}^{\infty} \frac{2^{g+d}}{j2^j} \frac{(-1)^{t+1} k^{t+1}}{(t-1)!(t+1)} \right) = R + \varepsilon$$

where $\varepsilon \leq \frac{1}{2^h}$ we can use the variant BBP algorithm here (involving the binary exponentiation algorithm) to ensure that $h = O(k \log k)$ we can take $h = 13g(2^g) + g13 + 10$ where g for computing the 2^g binary digit of γ - for a concrete examples .In the standard BBP algorithm it is known that the error can be made to be geometrically small by computing a known number of terms-in other words there are an infinite number of terms in the known BBP algorithm that total less than a geometric amount of absolute error- and we use this idea here. The error in the BBP formula occurs after ignoring all the term from s in it, the position

of s will be after the decimal point/base b point of the constant c

$$c = \sum_{u=0}^{\infty} \frac{p(u)}{q(u)b^u} = \sum_{u=0}^{s-1} \frac{p(u)}{q(u)b^u} + \sum_{u=s}^{\infty} \frac{p(u)}{q(u)b^u}$$

thus the error ϵ is

$$\epsilon = \left(\sum_{u=s}^{\infty} \frac{p(u)}{q(u)b^u} \right) \bmod 1$$

and R is

$$R = \left(\sum_{u=0}^{s-1} \frac{p(u)}{q(u)b^k} \right) \bmod 1$$

In other words to compute the n th bit of γ with our algorithm this involves computing at least $13g(2^g) + 13g + 10$ or

$O(k \log k)$ bits of $\ln k$ after the binary point of $\ln k$ using a variant BBP algorithm. There are two errors to consider in our digits extraction algorithm : the error for equation 1 which is known and the error from the variant of the BBP algorithm we use. Note that the bound $h = 13g(2^g) + g13 + 10$ is not tight so the space complexity and time complexiy of our algorithm will actually be better with a tighter bound. We can use integer arithmetic,used to implement arithmetic with rational fractions, to sum the h terms after the binary point of the constant $\ln 2^g$ and sum the other terms,equal to R , before the binary point of the constant $\ln 2^g$. We think this is feasible on a constrained embedded system, since from 3 as it only requires one division and one exponentation and implementing summation-this would involve implementing the mod function on arbitrary size integers which would take a lot less resources than support for floating point arithmetic and floating point values and floating point support for transcendental functions. Equation 1 would also require implementing arithmetic on rational fractions with arbitrary size integers and without the need for such floating point support mentioned above, again this would be feasible on a constrained embedded system. Of course, standard floating point arithmetic is a type of binary based fraction but the sort of implementation for rational fraction we consider would take less resources than for fully implementing the standard floating point arithmetic.

Now let analyse the error in the variant BBP algorithm to show that it is not too large and to show that γ digits are computed without error,

substituting into 10 gives

$$\begin{aligned}
&= \sum_{j=1}^{\infty} \left(\frac{2^{g+j}}{j 2^j} \right) \sum_{r=1}^{12k+1} \frac{(-1)^{r+1} k^{r+1}}{(r-1)!(r+1)} \bmod 1 = \\
&((R + \varepsilon) \frac{(-1)^{1+1} k^{1+1}}{(1-1)!(1+1)}) \bmod 1 + (R + \varepsilon) \frac{(-1)^{2+1} k^{2+1}}{(2-1)!(2+1)} \bmod 1 + \\
&(R + \varepsilon) \frac{(-1)^{3+1} k^{3+1}}{(3-1)!(3+1)} \bmod 1 \dots + \\
&(R + \varepsilon) \frac{(-1)^{(12k+1)+1} k^{(12k+1)+1}}{((12k+1)-1)!((12k+1)+1)}) \bmod 1
\end{aligned} \tag{11}$$

thus the error from the variant BBP algorithm, involving ϵ above and not the terms involving R above (these terms involving R are computed exactly from using arbitrary precision arithmetic to $O(13g(2^g) + 13g + 10)$ binary digits). So back to analysing the terms involving ϵ , this means the error here is

$$\begin{aligned}
&((\varepsilon) \frac{(+1)^{1+1} k^{1+1}}{(1-1)!(1+1)}) \bmod 1 + (\varepsilon) \frac{(+1)^{2+1} k^{2+1}}{(2-1)!(2+1)} \bmod 1 + \\
&(\varepsilon) \frac{(+1)^{3+1} k^{3+1}}{(3-1)!(3+1)} \bmod 1 \dots + \\
&(\varepsilon) \frac{(+1)^{(12k+1)+1} k^{(12k+1)+1}}{((12k+1)-1)!((12k+1)+1)}) \bmod 1
\end{aligned} \tag{12}$$

which simplifies to

$$\begin{aligned}
&(\varepsilon) \frac{k^{1+1}}{(1-1)!(1+1)} \bmod 1 + (\varepsilon) \frac{k^{2+1}}{(2-1)!(2+1)} \bmod 1 + \\
&(\varepsilon) \frac{k^{3+1}}{(3-1)!(3+1)} \bmod 1 \dots + \\
&(\varepsilon) \frac{k^{(12k+1)+1}}{((12k+1)-1)!((12k+1)+1)}) \bmod 1
\end{aligned} \tag{13}$$

now note for ϵ the mod 1 in each term can be removed as it we can make it close enough to zero where $\epsilon = 2^{-(13g(2^g) + g13 + 10)}$. Then we have

$$\begin{aligned}
&(\varepsilon) \frac{k^{1+1}}{(1-1)!(1+1)} + (\varepsilon) \frac{k^{2+1}}{(2-1)!(2+1)} + \\
&(\varepsilon) \frac{k^{3+1}}{(3-1)!(3+1)} \dots + \\
&(\varepsilon) \frac{k^{(12k+1)+1}}{((12k+1)-1)!((12k+1)+1)}) \bmod 1
\end{aligned} \tag{14}$$

$\implies$

$$\begin{aligned} & ((2^{-(13g(2^g)+g13+10)}) \frac{k^{1+1}}{(1-1)!(1+1)}) + (2^{-(13g(2^g)+g13+10)}) \frac{k^{2+1}}{(2-1)!(2+1)} \\ & (2^{-(13g(2^g)+g13+10)}) \frac{k^{3+1}}{(3-1)!(3+1)} \dots + \\ & (2^{-(13g(2^g)+g13+10)}) \frac{k^{(12k+1)+1}}{((12k+1)-1)!((12k+1)+1)}) \mod 1 \end{aligned} \quad (15)$$

and from and $k = 2^g$ we have

$$\begin{aligned} & ((2^{-(13g(2^g)+g13+10)}) \frac{(2^g)^{1+1}}{(1-1)!(1+1)}) + (2^{-(13g(2^g)+g13+10)}) \frac{(2^g)^{2+1}}{(2-1)!(2+1)} \\ & (2^{-(13g(2^g)+g13+10)}) \frac{(2^g)^{3+1}}{(3-1)!(3+1)} \dots + \\ & (2^{-(13g(2^g)+g13+10)}) \frac{(2^g)^{(12(2^g)+1)+1}}{((12(2^g)+1)-1)!((12(2^g)+1)+1)}) \mod 1 \end{aligned} \quad (16)$$

$\implies$

$$\begin{aligned} & ((2^{-(13g(2^g)+g13+10)}) \frac{(2^g)^2}{(1-1)!(1+1)}) + (2^{-(13g(2^g)+g13+10)}) \frac{(2^g)^3}{(2-1)!(2+1)} \\ & (2^{-(13g(2^g)+g13+10)}) \frac{(2^g)^4}{(3-1)!(3+1)} \dots + \\ & (2^{-(13g(2^g)+g13+10)}) \frac{(2^g)^{(12(2^g)+13)}}{((12(2^g)+1)-1)!((12(2^g)+1)+1)}) \mod 1 \end{aligned} \quad (17)$$

$\implies$

$$\begin{aligned} & ((2^{-(13g(2^g)+g13+10)}) \frac{(2^{2g})}{(1-1)!(1+1)}) + (2^{-(13g(2^g)+g13+10)}) \frac{(2^{3g})}{(2-1)!(2+1)} \\ & (2^{-(13g(2^g)+g13+10)}) \frac{(2^{4g})}{(3-1)!(3+1)} \dots + \\ & (2^{-(13g(2^g)+g13+10)}) \frac{(2^{(d12(2^d)+d13)})}{((12(2^g)+1)-1)!((12(2^g)+1)+1)}) \mod 1 \end{aligned} \quad (18)$$

$\implies$

$$\begin{aligned} & ((2^{-(13g(2^g)+g11+10)}) \frac{(1)}{(1-1)!(1+1)}) + (2^{-(13g(2^g)+g10+10)}) \frac{(1)}{(2-1)!(2+1)} \\ & (2^{-(13g(2^g)+g9+10)}) \frac{(1)}{(3-1)!(3+1)} \dots + \\ & (2^{-(g2^g)}) \frac{(1)}{((12(2^g)+1)-1)!((12(2^g)+1)+1)}) \mod 1 \end{aligned} \quad (19)$$

Since each $12k + 1$ terms above is bounded above by $2^{g^{2^g}}$ this means 19 is bounded above by

$$\begin{aligned} & \frac{12k + 1}{2^{g^{2^g}}} \\ &= \frac{12(2^g) + 1}{2^{g^{2^g}}} \end{aligned}$$

which for $g \geq 3$ gives

$$\frac{12(2^g) + 1}{2^{g^{2^g}}} < \frac{1}{2}$$

thus the binary digits of γ are correctly computed using our digit extraction algorithm from digit 2^g or 8th or more binary digit of γ but we also need to consider the error term of equation 1, apart from the error term from the variant BBP algorithm, which is $O(2^{-k})$ and the error term of 1 is

$$2k^k e^{-k}$$

to be more precise so when computing the 2^g digit will involve an error of

$$2k^{2^g} e^{-2^g}$$

thus computing binary digits start from position 52 after the binary point ,the error will be low enough then, for the binary digit of γ to be computed correctly.

3 A (Sketched) Dynamical Chaotic System Related to the Digits of γ

In [3] it is shown that the open question of proving the normality of π and $\ln 2$ is equivalent to the behaviour of a dynamical system and they refer to this as Hypothesis A in [3]. The dynamical system in [3] is derived from a BBP formula for π or $\ln 2$. In a related way we can also derive a dynamical system, from our digits extraction algorithm for γ or equation 8, and relate its behaviour of the digits of γ and thus give a potential new way to prove that γ is irrational. It is unknown if γ is irrational or rational. To do this involving rewriting the terms of 9.1 in an iterative way to derive a dynamical system and our version of Hypothesis A which is a hypothesis(or conjecture) relating the digits of γ and its normality and its plausible irrationality to the above sketched out derivation of the dynamical system relating to γ .

4 Another (Sketched) of Digit Extraction Algorithm for γ

We can also derive a digit extraction algorithm (we sketch out the algorithm here), and a digit related dynamical system, for γ , similar to above using one of Brent et al equation [5] such as

$$\gamma = \frac{A_w}{B_w} - \log(w) + O(e^{-4w}) \quad (20)$$

where

$$A_w = \sum_{q=0}^{\text{Ceil}(\delta w)} \left(\frac{n^w}{q!}\right)^2 H_k, B_w = \sum_{q=0}^{\text{Ceil}(\delta w)} \left(\frac{n^w}{q!}\right)^2$$

$$H_t = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{t}$$

where δ satisfies $\delta(\ln(\delta) - 1) = 1$

and Ceil is the integer Celing function

by considering BBP formula which exist for the value of w in equation 20 we compute

$$(B^D(\frac{A_n}{B_n} - \log(n))) \bmod 1$$

to compute the D th digit of γ in base B (B need not be binary here it can be decimal or $B = 10$ for example). The value of w in to determines what base the digits of γ can be extracted in. Arbitrary precision arithmetic is used which is up to $O(d^2)$ space and $O(d^2)$ time and precision d bits to compute the d decimal digit of binary digit of γ . There are more equation like 20 that can be use such as one that has terms

$$F_w = \frac{1}{4n} \sum_{q=0} \frac{((2q)!)^3}{(q!)^4 (16w)^{2q}}$$

used in a refined version of equation which is equation 21 below

$$\gamma = \frac{A_w}{B_w} - \frac{C_w}{B_w} - \log(w) + O(e^{-8w}) \quad (21)$$

but the delta in the upper summation index is given by

$$\delta(\ln(\delta) - 1) = 3$$

instead. Equations 20 and 21 have the right growth for the error terms for use as a similarly derived digit extraction algorithm for γ for binary

digits or decimal digits. Though the error term in 21 is from empirical evidence and is not proven.

The major disadvantage of using this equation as part of a digit extraction algorithm for γ is it involves the computation of δ computing transcendental functions which takes up a lot of resources on a constrained embedded system (possible running out of memory); such systems do not have hardware support for floating point numbers or hardware support support for operations for floating point numbers. Though it has this disadvantage, the sketched algorithm is still a new way to extract digits of γ and from that perspective it is interesting. The algorithm given in the previous section has a simpler (and thus less demanding in resources) for computing the upper limit on its summation there-it is of the for $12k + 1$, which takes significantly less embedded CPU/embedded MCU time and storage space to compute. For example the number $12k + 1$ can be computed in less than 50 bytes on an Intel 8008 CPU for suitable k . In a future paper we will investigate these more resource hungry digit extraction algorithms for γ . Implementing floating point arithmetic on a CPU such as the Intel 8080 would take much order of magnitudes more memory than 50 bytes.

5 References

- [1] Locate ref
- [2] Locate ref
- [3] Locate ref
- [4] Locate ref
- [5] Locate ref